



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

PROJECT REPORT

Subject: Programming For Bioinformatics (SECB3202)

Session: 2025/2026 Semester 1

Title: Pancreatic Cancer Prediction

Group Name	Group 10
GitHub Link	https://github.com/welsonxw/Pancrease_Cancer_Predictio_n
Group Members	1. RAVINESH A/L MARAN (A23CS0175) 2. WELSON WOONG LU BIN (A23CS0196) 3. BERNICE LOU MIN YUN (A23CS0056)
Section	01
Lecturer's Name	DR. SEAH CHOON SEN

Table of Content

1.0 Introduction	3
1.1 Problem Background	3
1.2 Problem Statement	4
1.3 Objectives	4
1.4 Scopes	4
2.0 Data collection and pre-processing	5
2.1 Importing Dataset	5
2.1.1 Understanding the Dataset	5
2.1.2 Importing and Exporting Data in Python	5
2.1.3 Getting started and analyzing data in Python	6
2.1.4 Python Packages for Data Science	8
2.2 Data Wrangling (Pandas/ Numpy)	9
2.2.1 Identifying and handling missing value	9
2.2.2 Data Formatting	11
2.2.3 Data Normalization (centering/ scaling)	12
2.2.4 Binning	13
2.2.5 Indicator Variables	15
2.3 Software and Hardware Requirements	18
3.0 Flowchart	19
3.1 Exploratory Data Analysis (EDA)	21
3.1.1 Descriptive Statistic	21
3.1.2 Basic Of Grouping	24
3.1.3 ANOVA	26
3.1.4 Correlation	28
3.2 Model Development	30
3.2.1 Simple and Multiple Linear Regression	31
3.2.2 Model evaluation using visualization	33
3.2.3 Polynomial regression and pipelines	35
3.2.4 R-squared and MSE for In-Sample Evaluation	37
3.2.5 Prediction and decision making	38
3.3 Model Evaluation (sklearn)	39
3.3.1 Model Evaluation	39
3.3.2 Over-fitting, under-fitting, and model selection	42
3.3.3 Ridge Regression	43
3.3.4 Grid Search	45
3.3.5 Model Refinement	47
4.0 Testing and Validation	48
5.0 Conclusion	50

1.0 Introduction

Pancreatic cancer is one of the most lethal human cancers; it is characterized by late diagnosis and rapid disease progression, leading to very poor survival rates. Symptoms are often subtle or mistaken for common gastrointestinal conditions that make early detection difficult. The global statistics on cancer indicate that the 5-year survival rate for pancreatic cancer is less than 10%, mainly due to the absence of reliable methods for its screening and biomarkers.

Recently, bioinformatics and machine learning have become key tools to predict cancer risk and identify the most important clinical indicators. The analysis of patient medical, clinical, and lifestyle data by machine learning models can reveal hidden relationships that may improve early detection and risk assessment of cancer.

This work is designed to apply a bioinformatics approach in the prediction of pancreatic cancer progression, analysis of key biomarkers, and investigation into how demographic, clinical, and lifestyle variables may influence patient outcomes.

1.1 Problem Background

Pancreatic cancer usually develops silently, and it is very hard to clearly identify early symptoms. The risk factors-smoking habits, obesity, diabetes, hereditary history, and chronic pancreatitis-all contribute to the progression of cancer. These factors are all interconnected, so doctors may not be able to interpret each and every one of them. Large datasets including symptoms of patients, tumor stages, lifestyle habits, types of treatment, socioeconomic conditions, physical activity level, and alcohol consumption allow for deeper analysis. Using bioinformatics approaches, these patterns can be more systematically studied to help predict cancer stage and identify high-risk individuals.

1.2 Problem Statement

Although clinical data on pancreatic cancer patients is indeed available, there still lacks a bioinformatics model that can automatically handle several parameters concerning a patient in order to predict the severity of cancer. The health expert might face difficulties in interpreting such high-dimensional data and understanding which factors most contribute to disease conditions. This leads to poor early detection, and as a result, patients usually get diagnosed when the disease has progressed to a higher stage. This calls for the development of a machine-learning-based bioinformatics model which can predict cancer stage or survival likelihood and identify the most influential risk factors.

1.3 Objectives

The main objective of this project is to develop a simple machine-learning prediction model for pancreatic cancer using clinical and lifestyle data. This project aims to preprocess and explore the dataset to identify important patterns, apply suitable classification methods to predict cancer stage or outcomes, and determine which factors act as key biomarkers. Another objective is to evaluate the performance of the model using common metrics such as accuracy or confusion matrix so that the reliability of the prediction can be understood.

1.4 Scopes

This project will focus on the Pancreatic Cancer Prediction Dataset from Kaggle. The scope includes cleaning the data, handling missing values, encoding categorical data, and performing simple exploratory analysis. The project will then apply selected machine learning classification techniques using Python libraries such as Pandas, NumPy, and Scikit-Learn. Only computational analysis will be performed; no real clinical diagnosis or laboratory experiments are included. The project also does not involve genomic or deep-learning-based approaches. All progress will be documented on GitHub as required by the course.

2.0 Data collection and pre-processing

2.1 Importing Dataset

2.1.1 Understanding the Dataset

The dataset used in this project is a Pancreatic Cancer Prediction dataset, which contains biomarker measurements and patient-related attributes relevant to disease classification. The dataset is intended to support the prediction of pancreatic cancer using machine learning techniques.

Before proceeding with data wrangling and model development, it is essential to understand the dataset structure, data types, and completeness. This step allows early identification of potential issues such as missing values, inconsistent data types, and abnormal distributions that may affect subsequent analysis.

2.1.2 Importing and Exporting Data in Python

The dataset was stored in CSV format and imported into the Python environment using the Pandas library within the Visual Studio Code platform. Pandas provides efficient functions for loading, handling, and manipulating structured data, making it suitable for bioinformatics-related datasets.

Importing the dataset into a Pandas DataFrame enables flexible data inspection, preprocessing, and transformation prior to analysis.

```
df = pd.read_csv(r"C:\bioinfo2\p4b\project\Pancrease_Cancer_Prediction\pancreatic_cancer_prediction_sample.csv")
df.head()
```

```
df = pd.read_csv(r"C:\bioinfo2\p4b\project\Pancrease_Cancer_Prediction\pancreatic_cancer_prediction_sample.csv")
df.head()
```

	Country	Age	Gender	Smoking_History	Obesity	Diabetes	Chronic_Pancreatitis	Family_History	Hereditary_Condition	Jaundice	...	Stage_at_Diagnosis	Survival_Time_Months	Treatment
0	Canada	64	Female	0	0	0	0	0	0	0	...	Stage III	13	?
1	South Africa	77	Male	1	1	0	0	0	0	0	...	Stage III	13	Chemotherapy
2	India	71	Female	0	0	0	0	0	0	0	...	Stage IV	3	Chemotherapy
3	Germany	56	Male	0	0	0	0	1	0	1	...	Stage IV	6	Radiotherapy
4	United States	82	Female	0	0	0	0	1	0	0	...	Stage IV	9	Chemotherapy

5 rows x 24 columns

2.1.3 Getting started and analyzing data in Python

After importing the dataset, an initial analysis was conducted to gain an overview of both numerical and categorical features using Python. Descriptive statistical methods were applied to summarise the data and to understand the distribution, variability, and characteristics of the dataset before proceeding with further preprocessing and modeling.

```
df.describe().T
```

✓ 0.0s

	count	mean	std	min	25%	50%	75%	max
Age	50000.0	64.54094	9.973847	30.0	58.0	65.0	71.0	90.0
Smoking_History	50000.0	0.29954	0.458061	0.0	0.0	0.0	1.0	1.0
Obesity	50000.0	0.24826	0.432008	0.0	0.0	0.0	0.0	1.0
Diabetes	50000.0	0.19998	0.399989	0.0	0.0	0.0	0.0	1.0
Chronic_Pancreatitis	50000.0	0.09930	0.299067	0.0	0.0	0.0	0.0	1.0
Family_History	50000.0	0.15168	0.358714	0.0	0.0	0.0	0.0	1.0
Hereditary_Condition	50000.0	0.04944	0.216787	0.0	0.0	0.0	0.0	1.0
Jaundice	50000.0	0.19922	0.399418	0.0	0.0	0.0	0.0	1.0
Abdominal_Discomfort	50000.0	0.29650	0.456719	0.0	0.0	0.0	1.0	1.0
Back_Pain	50000.0	0.25286	0.434656	0.0	0.0	0.0	1.0	1.0
Weight_Loss	50000.0	0.34998	0.476968	0.0	0.0	0.0	1.0	1.0
Development_of_Type2_Diabetes	50000.0	0.19622	0.397141	0.0	0.0	0.0	0.0	1.0
Survival_Time_Months	50000.0	13.89804	11.272151	1.0	6.0	10.0	19.0	59.0
Survival_Status	50000.0	0.12844	0.334582	0.0	0.0	0.0	0.0	1.0
Alcohol_Consumption	50000.0	0.30346	0.459757	0.0	0.0	0.0	1.0	1.0

The `df.describe()` function was used to generate descriptive statistics for numerical variables in the dataset. This includes measures such as count, mean, standard deviation, minimum and maximum values, as well as quartiles (25%, 50%, and 75%).

From the results, the dataset contains 50,000 records, indicating a large and reliable sample size. The average age of patients is approximately 64.5 years, with most patients falling between 58 and 71 years, suggesting that pancreatic cancer predominantly affects older individuals. Survival time statistics show a median of 10 months, reflecting the aggressive nature of the disease and generally low survival outcomes.

Several clinical and lifestyle factors, such as smoking history, obesity, diabetes, and alcohol consumption, are represented as binary indicators. The mean values of these variables represent their prevalence in the dataset. For example, smoking history and alcohol consumption are present in roughly 30% of patients. This numerical analysis helps identify the distribution of key risk factors within the population.

```
df.describe(include='object').T
```

✓ 0.0s

	count	unique	top	freq
Country	50000	9	United States	17608
Gender	50000	2	Male	25962
Stage_at_Diagnosis	50000	4	Stage IV	19922
Treatment_Type	50000	3	Chemotherapy	24910
Physical_Activity_Level	50000	3	Medium	20038
Diet_Processed_Food	50000	3	Medium	20122
Access_to_Healthcare	50000	3	Medium	25268
Urban_vs_Rural	50000	2	Urban	35003
Economic_Status	50000	3	Middle	24881

The `df.describe(include='object')` function was applied to analyse categorical variables by examining their counts, number of unique categories, most frequent values, and frequency of occurrence.

The results show that the majority of patients are from the United States, and male patients slightly outnumber female patients. A large proportion of patients are diagnosed at Stage IV, indicating late-stage detection of pancreatic cancer. Chemotherapy is the most commonly recorded treatment type, which aligns with clinical practice for advanced-stage cases.

Lifestyle and socio-economic variables, including physical activity level, diet, access to healthcare, residential setting, and economic status, are predominantly classified as “Medium,” suggesting moderate lifestyle and healthcare conditions among most patients.

This categorical analysis provides valuable insight into the demographic and clinical composition of the dataset.

2.1.4 Python Packages for Data Science

Several Python libraries were utilized during this phase of the project:

- Pandas – data loading, manipulation, and preprocessing
- NumPy – numerical computations and array operations
- Matplotlib – basic data visualization
- Seaborn – statistical data visualization
- Scikit-learn – data normalization and preparation for machine learning

These libraries collectively support efficient data analysis, preprocessing, and preparation for predictive modeling.

2.2 Data Wrangling (Pandas/ Numpy)

2.2.1 Identifying and handling missing value

This code is used to examine the dataset for missing values and duplicate records. The function `df.isna().mean() * 100` calculates the percentage of missing values for each column, allowing verification of data completeness. The results show that all variables contain 0% missing values, indicating that no null data is present.

The function `df.duplicated()` is used to determine the proportion of duplicate rows in the dataset. Although the percentage of duplicate records is very small, the total number of duplicate rows is identified using `df.duplicated().sum()`

```
▶ ✓  
# missing value and duplicate value check  
print('Missing Value (%)')  
print(df.isna().mean()*100)  
print('\nDuplicate Row (%)')  
print(df.duplicated().mean())  
[18] ✓ 0.0s
```

```
Missing Value (%)  
Country                0.0  
Age                    0.0  
Gender                 0.0  
Smoking_History        0.0  
Obesity                0.0  
Diabetes               0.0  
Chronic_Pancreatitis   0.0  
Family_History         0.0  
Hereditary_Condition   0.0  
Jaundice               0.0  
Abdominal_Discomfort   0.0  
Back_Pain              0.0  
Weight_Loss            0.0  
Development_of_Type2_Diabetes 0.0  
Stage_at_Diagnosis     0.0  
Survival_Time_Months   0.0  
Treatment_Type         0.0  
Survival_Status        0.0  
Alcohol_Consumption    0.0  
Physical_Activity_Level 0.0  
Diet_Processed_Food    0.0  
Access_to_Healthcare   0.0  
Urban_vs_Rural         0.0  
Economic_Status        0.0  
dtype: float64  
  
Duplicate Row (%)  
8e-05
```

To ensure data consistency, duplicate rows are removed using the `drop_duplicates()` function. The output confirms that four duplicate rows were detected before removal and that no duplicate rows remain after cleaning. This process ensures that the dataset is clean and ready for further preprocessing and analysis.

```
print("Number of duplicate rows BEFORE removal:", df.duplicated().sum())
df.drop_duplicates(inplace=True)
print("Number of duplicate rows AFTER removal:", df.duplicated().sum())
```

✓ 0.0s

```
...  Number of duplicate rows BEFORE removal: 4
      Number of duplicate rows AFTER removal: 0
```

2.2.2 Data Formatting

This code standardises column names and categorical text values to ensure consistency and prevent errors during data processing. All column names are converted to lowercase using `df.columns.str.lower()` to maintain a uniform naming convention throughout the dataset.

The values in the `gender` column are converted to lowercase using string operations to eliminate inconsistencies caused by different text formats. Additionally, spaces in the `stage_at_diagnosis` column are replaced with underscores to create standardized category labels suitable for further encoding and analysis.

The printed output confirms that column names have been successfully standardised, the `gender` variable contains only two consistent categories (female and male), and the `stage_at_diagnosis` variable has been properly formatted into uniform categories (Stage_I, Stage_II, Stage_III, and Stage_IV). This data formatting step ensures that categorical variables are clean, consistent, and ready for encoding and machine learning processing.

```
# standardise column names and text values for stage_at_diagnosis
df.columns = df.columns.str.lower()
df['gender'] = df['gender'].str.lower()
df['stage_at_diagnosis'] = df['stage_at_diagnosis'].str.replace(' ', '_')

print(df.columns)
print(df['gender'].unique())
print(df['stage_at_diagnosis'].unique())
```

✓ 0.0s

```
Index(['country', 'age', 'gender', 'smoking_history', 'obesity', 'diabetes',
      'chronic_pancreatitis', 'family_history', 'hereditary_condition',
      'jaundice', 'abdominal_discomfort', 'back_pain', 'weight_loss',
      'development_of_type2_diabetes', 'stage_at_diagnosis',
      'survival_time_months', 'treatment_type', 'survival_status',
      'alcohol_consumption', 'physical_activity_level', 'diet_processed_food',
      'access_to_healthcare', 'urban_vs_rural', 'economic_status'],
      dtype='object')
['female' 'male']
['Stage_III' 'Stage_IV' 'Stage_II' 'Stage_I']
```

2.2.3 Data Normalization (centering/ scaling)

In this step, data normalization was applied to selected numerical features to ensure they are on a comparable scale before machine learning analysis. The StandardScaler from the Scikit-learn library was used to standardize the age and survival_time_months variables.

The fit_transform() function rescales the data by subtracting the mean and dividing by the standard deviation, resulting in features with a mean close to 0 and a standard deviation close to 1. This process helps prevent features with larger numeric ranges from dominating the learning process.

The descriptive statistics shown in the output confirm successful normalization, where both variables exhibit mean values approximately equal to zero and standard deviations close to one. This indicates that the numerical features have been properly centered and scaled, making them suitable for use in machine learning models.

```
# data normalization
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df[['age', 'survival_time_months']] = scaler.fit_transform(
    df[['age', 'survival_time_months']]
)

# actual
print(df[['age', 'survival_time_months']].describe())

# rounded
desc = df[['age', 'survival_time_months']].describe()
desc.loc['count'] = desc.loc['count'].astype(int)
desc.round(10)
```

The results show that the normalization process was successfully applied to the age and survival_time_months variables. After standardization, both variables have mean values approximately equal to zero and standard deviations close to one, which confirms effective centering and scaling.

The minimum and maximum values indicate that the normalized data now represents how far each observation deviates from the mean in terms of standard deviation units. For example, negative values represent observations below the mean, while positive values represent observations above the mean.

Overall, the normalized results demonstrate that numerical features have been placed on a consistent scale, reducing the influence of differing units and magnitudes. This improves the stability and performance of machine learning algorithms in subsequent modeling stages.

	age	survival_time_months
count	4.999600e+04	4.999600e+04
mean	3.612688e-16	-4.093054e-17
std	1.000010e+00	1.000010e+00
min	-3.463106e+00	-1.144306e+00
25%	-6.558025e-01	-7.007351e-01
50%	4.602341e-02	-3.458782e-01
75%	6.475885e-01	4.525496e-01
max	2.552545e+00	4.001118e+00

	age	survival_time_months
count	49996.000000	49996.000000
mean	0.000000	-0.000000
std	1.000010	1.000010
min	-3.463106	-1.144306
25%	-0.655803	-0.700735
50%	0.046023	-0.345878
75%	0.647588	0.452550
max	2.552545	4.001118

2.2.4 Binning

In this step, binning was applied to the age variable to group continuous values into meaningful categories. Since the age feature had been previously standardized using StandardScaler, the bin boundaries were defined based on scaled values.

The `pd.cut()` function was used to divide the normalized age values into three categories: Young, Middle-aged, and Elderly. Each data point was assigned to a corresponding age group based on its standardized value. This transformation converts a continuous numerical feature into a categorical variable, making it easier to analyze age-related patterns.

The output confirms that the binning process was applied successfully. Sample rows show the correct assignment of age values to their respective age groups. The frequency count indicates that the majority of patients fall into the Elderly category, followed by Middle-aged, while the Young group represents the smallest proportion of the dataset.

Overall, binning simplifies data representation and can help highlight age-related trends in pancreatic cancer prevalence during further analysis.

```
# binning age column
df['age_group'] = pd.cut(
    df['age'],
    bins=[-3, -1, 0, 3], # scaled values after StandardScaler
    labels=['Young', 'Middle-aged', 'Elderly']
)

print(df[['age', 'age_group']].head(10))
print(df['age_group'].value_counts())
```

[47] ✓ 0.0s

```

      age  age_group
0 -0.054237 Middle-aged
1  1.249154      Elderly
2  0.647588      Elderly
3 -0.856324 Middle-aged
4  1.750458      Elderly
5 -1.558150      Young
6  0.246545      Elderly
7 -0.856324 Middle-aged
8 -1.457889      Young
9 -1.157107      Young
age_group
Elderly      25158
Middle-aged   16994
Young         7776
Name: count, dtype: int64

```

2.2.5 Indicator Variables

In this step, categorical variables were converted into indicator variables to make them suitable for machine learning models. The `pd.get_dummies( )` function was used to perform one-hot encoding on selected categorical features, including `gender`, `treatment_type`, `stage_at_diagnosis`, and `urban_vs_rural`

The parameter `drop_first=True` was applied to avoid multicollinearity by removing one reference category from each encoded variable. This ensures that the resulting dataset does not contain redundant features that could negatively affect model performance.

```

# indicator variable
df_encoded = pd.get_dummies(
    df,
    columns=['gender', 'treatment_type', 'stage_at_diagnosis', 'urban_vs_rural'],
    drop_first=True
)

print(df_encoded.columns)
print(df_encoded.head())

```

[48] ✓ 0.0s

The output shows that each category has been transformed into a separate binary column with values of True or False, indicating the presence or absence of a specific category. For example, variables such as `gender_male`, `treatment_type_Surgery`, and `stage_at_diagnosis_Stage_IV` represent individual category indicators.

Overall, this transformation enables categorical information to be represented numerically, ensuring compatibility with machine learning algorithms and preparing the dataset for model development in subsequent project stages.

```

Index(['country', 'age', 'smoking_history', 'obesity', 'diabetes',
      'chronic_pancreatitis', 'family_history', 'hereditary_condition',
      'jaundice', 'abdominal_discomfort', 'back_pain', 'weight_loss',
      'development_of_type2_diabetes', 'survival_time_months',
      'survival_status', 'alcohol_consumption', 'physical_activity_level',
      'diet_processed_food', 'access_to_healthcare', 'economic_status',
      'age_group', 'gender_male', 'treatment_type_Radiation',
      'treatment_type_Surgery', 'stage_at_diagnosis_Stage_II',
      'stage_at_diagnosis_Stage_III', 'stage_at_diagnosis_Stage_IV',
      'urban_vs_rural_Urban'],
      dtype='object')

```

	country	age	smoking_history	obesity	diabetes	\
0	Canada	-0.054237	0	0	0	
1	South Africa	1.249154	1	1	0	
2	India	0.647588	0	0	0	
3	Germany	-0.856324	0	0	0	
4	United States	1.750458	0	0	0	

	chronic_pancreatitis	family_history	hereditary_condition	jaundice	\
0	0	0		0	0
1	0	0		0	0
2	0	0		0	0
3	0	1		0	1
4	0	1		0	0

	abdominal_discomfort	...	access_to_healthcare	economic_status	\
0	0	...	High	Low	
1	0	...	Medium	Low	
2	0	...	Low	Middle	
3	0	...	Medium	Middle	
4	0	...	Medium	Low	

	age_group	gender_male	treatment_type_Radiation	treatment_type_Surgery	\
0	Middle-aged	False	False	True	
1	Elderly	True	False	False	
2	Elderly	False	False	False	
3	Middle-aged	True	True	False	
4	Elderly	False	False	False	

	stage_at_diagnosis_Stage_II	stage_at_diagnosis_Stage_III	\
0	False	True	
1	False	True	
2	False	False	
3	False	False	
4	False	False	

	stage_at_diagnosis_Stage_IV	urban_vs_rural_Urban
0	False	True
1	False	True
2	True	False
3	True	False
4	True	False

[5 rows x 28 columns]

2.3 Software and Hardware Requirements

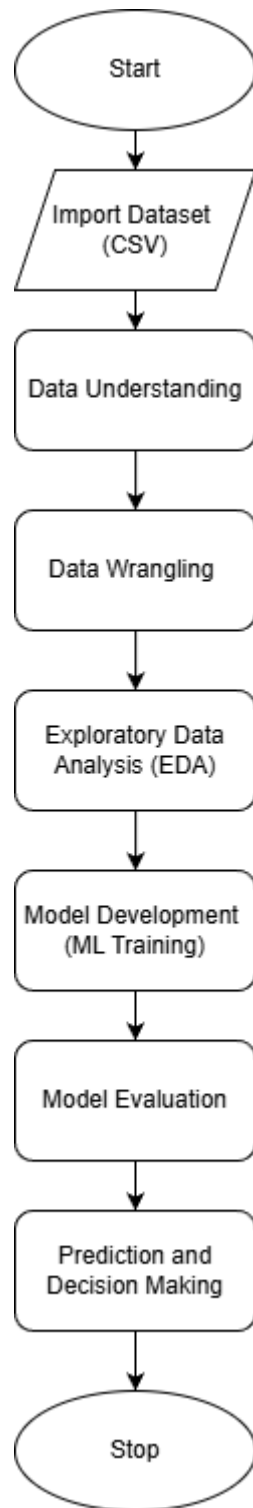
Software Requirement:

Software	Version	Purpose
Python	Python 3.12	Main programming language for the project
Pandas	Latest	Data manipulation and preprocessing
NumPy	Latest	Numerical operations
Matplotlib	Latest	Data visualization
Scikit - Learn	Latest	Machine learning model development and evaluation
Jupyter Notebook	Latest	Coding environment, testing and documentation
GitHub	N/A	Version control and project progress tracking
VS Code	Latest	Python IDE

Hardware Requirement:

Hardware	Minimum Requirement	Recommended
Laptop / PC	i3 / Ryzen 3	i5 / Ryzen 5
RAM	4 GB	8 GB and above
Storage	1 GB Free Space	1 GB Free Space or Higher
GPU	Not required	NVIDIA GPU (<i>optional for faster ML</i>)
Internet	Dataset download purpose	Stable Internet

3.0 Flowchart



Flowchart 1.0: Flow of the Project

The flowchart begins with importing the pancreatic cancer dataset in CSV format. Once the data is loaded, the next step is to understand the dataset by examining the variables, data types, and overall structure. After gaining an initial understanding, data wrangling is performed to handle missing values, encode categorical variables, and clean the dataset for further analysis.

Exploratory Data Analysis (EDA) is then conducted to observe patterns, correlations, and distributions within the data. Insights from EDA help guide the machine learning phase, where classification models are developed and trained. After the models are built, their performance is evaluated using appropriate metrics. The results from these evaluations form the basis of prediction and decision-making, concluding the overall workflow.

3.1 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to determine the patterns, relationships and trends within pancreatic cancer dataset. This step aims at summarising the key features of the data in terms of descriptive statistics, group-based analysis, inferential statistics, and correlation analysis. EDA offers valuable information on the relationship between patient survival outcomes and demographic, clinical and lifestyle variables.

3.1.1 Descriptive Statistic

In this stage, descriptive statistics were produced to generalize the major features of the numerical and categorical variables in the dataset. This analysis helps to get a preliminary idea about the distribution of data, central tendency, and variability, after which additional statistical analysis is conducted.

In the case of numerical variables, the `describe()` function was used to compute descriptive statistics of age and survival months. The two variables have 49,996 observations and that means that there are no missing values. The mean score of these variables is approximately 0, and the standard deviations are almost 1 because these variables are already standardized with the `StandardScaler`. This ascertains that the normalization process was successfully implemented. The lowest and the highest values show that variability and possible outliers are present, especially in the survival time that has a greater range of values.

In the case of categorical variables, the frequency distributions were created concerning the gender, stage at diagnosis, treatment type, smoking history and survival status. The gender distribution is rather balanced with both males and female patients having similar distributions. The distribution of stage at diagnosis reveals that the majority of the patients were diagnosed in their advanced stages with Stage IV being the largest with Stage III and Stage I coming in the least group. The trend indicates late diagnosis that is usually related to pancreatic cancer.

The variable survival status shows that there is an apparent imbalance of classes since most of the patients are identified as deceased as opposed to a significantly insignificant number of survivors. This imbalance corresponds to the aggressive character of pancreatic cancer and is

also a significant factor to be taken into account in the future modeling and assessment of the model.

In general, descriptive statistics will give the necessary information on the organization and peculiarities of the dataset and ensure the effective data preprocessing and crucial clinical trends that will be used in further exploratory and predictive analysis.

```
# descriptive statistic

desc_numeric = df[['age', 'survival_time_months']].describe()
print(desc_numeric)

#binary variable
categorical_cols = [
    'gender',
    'stage_at_diagnosis',
    'treatment_type',
    'smoking_history',
    'survival_status'
]

for col in categorical_cols:
    print(f"\n{col} distribution:")
    print(df[col].value_counts())
```

	age	survival_time_months
count	4.999600e+04	4.999600e+04
mean	3.612688e-16	-4.093054e-17
std	1.000010e+00	1.000010e+00
min	-3.463106e+00	-1.144306e+00
25%	-6.558025e-01	-7.007351e-01
50%	4.602341e-02	-3.458782e-01
75%	6.475885e-01	4.525496e-01
max	2.552545e+00	4.001118e+00

gender distribution:

gender

male 25959

female 24037

Name: count, dtype: int64

stage_at_diagnosis distribution:

stage_at_diagnosis

Stage_IV 19918

Stage_III 14968

Stage_II 10173

Stage_I 4937

Name: count, dtype: int64

treatment_type distribution:

...

survival_status

0 43574

1 6422

Name: count, dtype: int64

3.1.2 Basic Of Grouping

This step involved basic grouping to investigate the relationship that existed between various categorical variables and the variable, survival time months. The group means are the average z-scores as they represent the relative survival with respect to the entire dataset as the time to survive had been standardized before.

In the case of age groups, the data was clustered according to the already established categories (Below-Average, Average, Above-Average). These findings indicate that the patients in the Below-Average age and Above-Average age group have a slightly better survival score (0.0010 and 0.0020 respectively) than the Average group (-0.0032). Even though the differences are minor as a result of standardization, it implies that there is a slight variation in the survival outcomes of age groups.

When classified according to the stage of diagnosis, it is evident that there is a declining trend in terms of survival with cancer progression. Stage I patients had the best average survival (1.88) and Stage II (0.58), Stage III (-0.08) and Stage IV had the lowest survival (-0.70) respectively. This supports the anticipated clinical trend in that the earlier the diagnosis is made the better the survival.

On the type of treatment, Radiation (0.0073), Chemotherapy (0.0004), and Surgery (0.0122) patients report the best mean survival of the patients. Such differences raise the possibility of variation in treatments and the selection of patients.

Lastly, smoking history is purported to have a small impact on survival. The mean survival of patients with prior history of smoking (1) is marginally higher than it is in non-smokers (-0.0023), but the difference is small and it may need further testing to determine the significance.

In general, the simplified grouping analysis offers helpful information regarding the variation of survival according to the most significant patient characteristics and clinical variables. These trends can inform future statistical tests and inform the formulation of predictive models on pancreatic cancer survival.

```
# basic grouping
# Survival (months)=(z-score $\times\sigma$  (std))+  $\mu$ (mean)

age_group_survival = df.groupby('age_group', observed=True)['survival_time_months'].mean()
print(age_group_survival)

print("\n")
stage_survival = df.groupby('stage_at_diagnosis')['survival_time_months'].mean()
print(stage_survival)

print("\n")
treatment_survival = df.groupby('treatment_type')['survival_time_months'].mean()
print(treatment_survival)

print("\n")
smoking_survival = df.groupby('smoking_history')['survival_time_months'].mean()
print(smoking_survival)
```

```
age_group
Below-Average    0.001027
Average          -0.003200
Above-Average    0.002046
Name: survival_time_months, dtype: float64
```

```
stage_at_diagnosis
Stage_I          1.884412
Stage_II         0.576556
Stage_III       -0.082687
Stage_IV        -0.699417
Name: survival_time_months, dtype: float64
```

```
treatment_type
Chemotherapy     0.000430
Radiation        0.007325
Surgery          -0.012202
Name: survival_time_months, dtype: float64
```

```
smoking_history
0    -0.002257
1     0.005276
Name: survival_time_months, dtype: float64
```

3.1.3 ANOVA

In this step, ANOVA (Analysis of Variance) was done to determine whether there are statistically significant differences in survival time months between patients in various stages of diagnosis. The ANOVA will be suitable in this case since the independent variable (stage at diagnosis) has multiple categories whereas the dependent variable (survival time) is continuous.

The data was categorized based on stage at diagnosis, and the `scipy.stats` library was called on `f oneway` which was used to generate the F-statistic and p-value. The results are as follows:

- F-statistic: 26,644.67
- p-value: 0.0

The fact that F-statistic and p-value of 0.0 are high shows that the differences in the survival times between at least two diagnosis stages are highly significant. This is in line with clinical anticipation because at earlier stages (Stage I and II), patients have very long survival rate as compared to patients at later stages (Stage III and IV).

On the whole, the ANOVA proves that the stage of diagnosis is statistically significantly related to survival, which supports the necessity of early diagnosis as an effective measure of patient outcomes.

```

# anova

from scipy.stats import f_oneway

stage_groups = [
    df[df['stage_at_diagnosis'] == stage]['survival_time_months']
    for stage in df['stage_at_diagnosis'].unique()
]

f_stat_stage, p_value_stage = f_oneway(*stage_groups)

print("ANOVA: Survival Time vs Stage at Diagnosis")
print("F-statistic:", f_stat_stage)
print("p-value:", p_value_stage)

```

```

ANOVA: Survival Time vs Stage at Diagnosis
F-statistic: 26644.673839144936
p-value: 0.0

```

3.1.4 Correlation

Correlation analysis was conducted in this step to test the linearity between numerical variables in the data set and this may assist in determining the pattern or possible predictor of the outcome of survival.

Because the calculations of correlation need numerical inputs all the categorical variables were first transformed into numbers by using Label Encoding. Each distinct category of variables like gender, stage at diagnosis, type of treatment, and smoking history was assigned to a distinct integer value, and this enabled the variables to be incorporated in computation of correlation.

Then, standardization of continuous numerical variables was done by StandardScaler so that all variables are put on equal scale. This helps avoid having features with a bigger range to affect correlation measures disproportionately.

The dataset was also preprocessed and was then prepared to be correlated. The strong and directional linear relationships between variables are quantified using pairwise correlations which are usually measured using Pearson correlation coefficient. The values near 1 or -1 are strong positive or negative linear relationships respectively and the near values of 0 are weak or no linear relationship.

This preprocessing allows results of the correlation to be meaningful and comparable between features and gives information about which variables correlate with survival time months most strongly and adds information to future modeling and features to incorporate.

```
# correlation analysis

label_encoders = {}
for col in df.select_dtypes(include=['object']).columns:
    label_encoders[col] = LabelEncoder()
    df[col] = label_encoders[col].fit_transform(df[col])

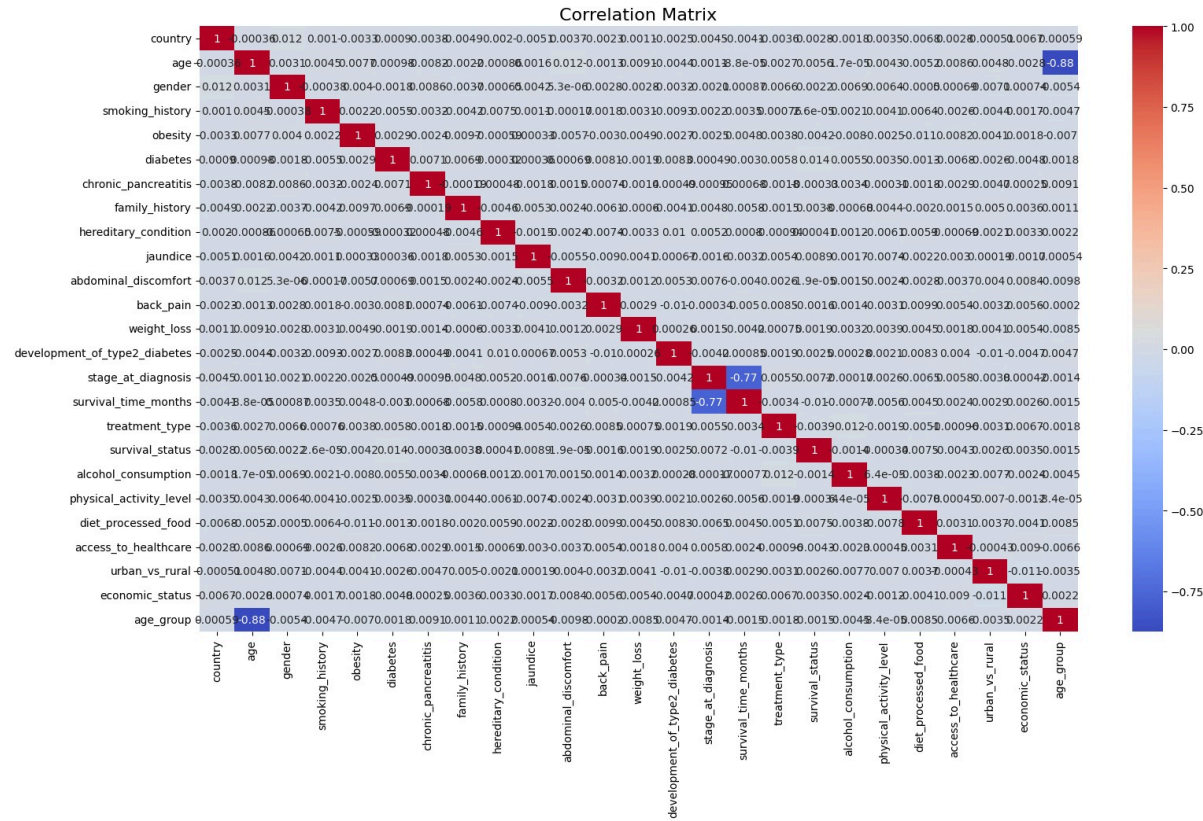
original_objects_cols = list(label_encoders.keys())
num_col = df.select_dtypes(include=['int64', 'float64']).columns

num_col_scale = [col for col in num_col if col not in original_objects_cols and col != 'Survival_Status']

scaler = StandardScaler()
df[num_col_scale] = scaler.fit_transform(df[num_col_scale])

df[num_col_scale].head()
```

age	smoking_history	obesity	diabetes	chronic_pancreatitis	family_history	hereditary_condition	jaundice	abdominal_discomfort	back_pain	weight_loss	development_of_type2_diabetes
-0.054237	-0.653974	-0.574702	-0.499994	-0.332013	-0.422868	-0.22807	-0.498806	-0.649208	-0.581754	-0.733748	-0.494111
1.249154	1.529113	1.740032	-0.499994	-0.332013	-0.422868	-0.22807	-0.498806	-0.649208	-0.581754	-0.733748	2.023836
0.647588	-0.653974	-0.574702	-0.499994	-0.332013	-0.422868	-0.22807	-0.498806	-0.649208	-0.581754	1.362866	2.023836
-0.856324	-0.653974	-0.574702	-0.499994	-0.332013	2.364804	-0.22807	2.004788	-0.649208	-0.581754	-0.733748	2.023836
1.750458	-0.653974	-0.574702	-0.499994	-0.332013	2.364804	-0.22807	-0.498806	-0.649208	-0.581754	-0.733748	-0.494111



3.2 Model Development

In this phase, machine learning models were developed to predict pancreatic cancer-related outcomes using the preprocessed dataset. Simple and multiple linear regression models were implemented to examine the relationship between selected features and the target variable, while polynomial regression was applied to capture potential non-linear patterns. Pipelines were used to streamline preprocessing and model training.

```
# prepare data regression

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X = df[['age']] # simple regression
y = df['survival_time_months']
```

Model performance was evaluated using visualization techniques and quantitative metrics such as R-squared and Mean Squared Error (MSE) for in-sample evaluation. These results were used to assess model accuracy and goodness of fit. Finally, the trained models were applied for prediction and basic decision-making, demonstrating how the developed models support data-driven analysis in pancreatic cancer studies.

3.2.1 Simple and Multiple Linear Regression

```
# simple linear regression

lin_reg = LinearRegression()
lin_reg.fit(X, y)

y_pred = lin_reg.predict(X)

mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)

print("Simple Linear Regression")
print("MSE:", mse)
print("R²:", r2)
```

The simple linear regression model was implemented using `LinearRegression()` from Scikit-learn. The model was trained with age as the independent variable (**X**) and survival_time_months as the dependent variable (**y**). After fitting the model to the data, predictions of survival time were generated using the `predict()` function.

To evaluate the model, Mean Squared Error (MSE) and R-squared (R^2) were calculated. These metrics measure prediction error and the proportion of variance in survival time explained by age.

```
Simple Linear Regression
MSE: 0.9999999922847456
R²: 7.715254435325392e-09
```

The output shows an MSE value close to 1 and an R^2 value very close to zero. This indicates that age alone explains almost none of the variation in survival time. The high prediction error and low explanatory power suggest that simple linear regression is insufficient for modeling survival outcomes using only age.

```

# multiple linear regression

X_multi = df[['age', 'smoking_history', 'diabetes', 'obesity']]

multi_reg = LinearRegression()
multi_reg.fit(X_multi, y)

y_pred_multi = multi_reg.predict(X_multi)

print("Multiple Linear Regression")
print("MSE:", mean_squared_error(y, y_pred_multi))
print("R²:", r2_score(y, y_pred_multi))
]

```

In multiple linear regression, the model was extended by including additional predictors: age, smoking history, diabetes, and obesity. These features were combined into a single input matrix (`X_multi`) and used to train a `LinearRegression` model. Predictions were then generated using the trained model.

The model performance was evaluated using the same metrics, MSE and R^2 , to allow direct comparison with the simple linear regression model.

```

Multiple Linear Regression
MSE: 0.9999559925981186
R²: 4.4007401881374086e-05

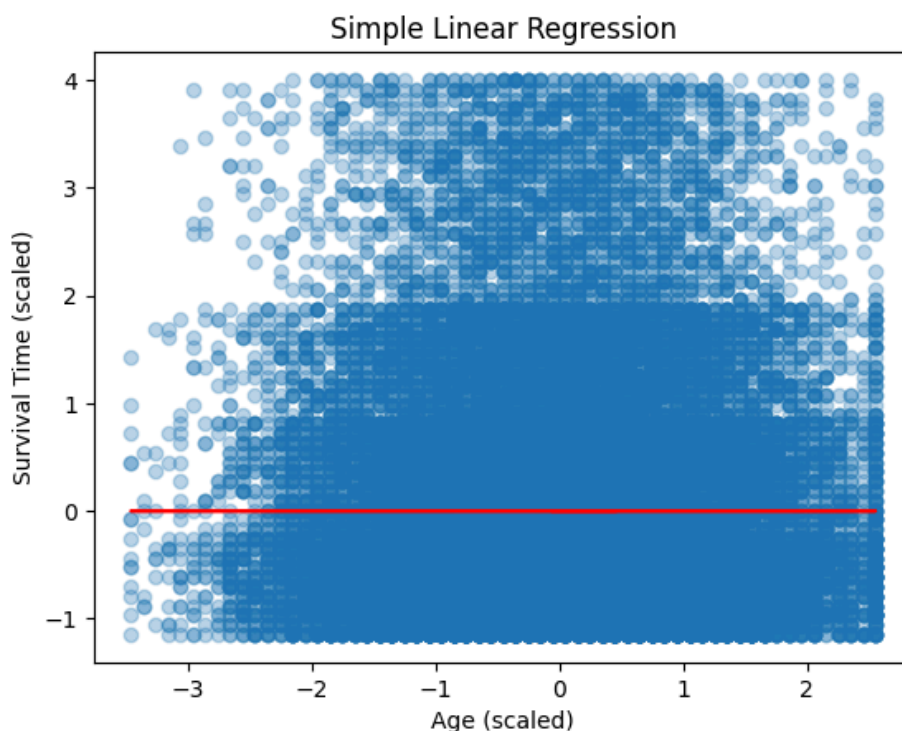
```

The results show a slightly improved R^2 value compared to simple linear regression, while the MSE remains close to 1. This indicates that adding more predictors provides limited improvement in explaining survival time. Although multiple variables were included, the model still has weak predictive power, highlighting the complexity of survival prediction and the need for more advanced modeling approaches.

3.2.2 Model evaluation using visualization

```
# visualization

plt.scatter(X, y, alpha=0.3)
plt.plot(X, y_pred, color='red')
plt.xlabel("Age (scaled)")
plt.ylabel("Survival Time (scaled)")
plt.title("Simple Linear Regression")
plt.show()
```



This visualization evaluates the performance of the simple linear regression model by comparing actual data points with the predicted regression line. A scatter plot is used to display the relationship between scaled age and scaled survival time, where each point represents an individual patient record.

The red line represents the predicted values generated by the linear regression model. The wide dispersion of data points around the regression line indicates a weak linear relationship between age and survival time. This visual observation supports the low R-squared value obtained earlier, confirming that age alone does not adequately explain variations in survival time.

Overall, this visualization provides a clear graphical assessment of model performance and highlights the limitation of simple linear regression, motivating the use of more complex models with additional features.

3.2.3 Polynomial regression and pipelines

Polynomial regression was implemented to model potential non-linear relationships between the independent variable and survival time. The `PolynomialFeatures(degree=2)` function was used to expand the original feature into polynomial terms, allowing the model to capture curved patterns in the data.

```
# polynomial regression

from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)

poly_model.fit(X, y)
y_pred_poly = poly_model.predict(X)

print("Polynomial Regression (degree=2)")
print("MSE:", mean_squared_error(y, y_pred_poly))
print("R²:", r2_score(y, y_pred_poly))
```

A pipeline was created using `make_pipeline()` to combine polynomial feature generation and linear regression into a single workflow. This ensures that feature transformation and model training are applied consistently and efficiently. The pipeline was then fitted to the dataset using `fit()`, and predictions were generated using the `predict()` function.

Model performance was evaluated using Mean Squared Error (MSE) and R-squared (R^2) to measure prediction error and explanatory power.

```
Polynomial Regression (degree=2)
MSE: 0.9999951778533491
R²: 4.822146650873549e-06
```

The output shows an MSE value close to 1 and an R^2 value very close to zero. This indicates that introducing polynomial terms does not significantly improve the model's ability to explain variations in survival time compared to simple and multiple linear regression.

The very low R^2 value suggests that even when non-linear relationships are considered, the selected predictor(s) remain weak in explaining survival outcomes. This result highlights that survival time is influenced by complex factors that are not adequately captured by polynomial regression using the current feature set.

3.2.4 R-squared and MSE for In-Sample Evaluation

In-sample evaluation was performed using R-squared (R^2) and Mean Squared Error (MSE) to assess the performance of the regression models on the training data.

R-squared (R^2) measures the proportion of variance in the target variable that is explained by the model. A higher R^2 value indicates better explanatory power, while values close to zero suggest that the model explains very little of the variation in survival time.

Mean Squared Error (MSE) measures the average squared difference between the actual and predicted values. Lower MSE values indicate better prediction accuracy, while higher values reflect larger prediction errors.

In this study, the models produced very low R^2 values and MSE values close to one, indicating limited predictive performance. These results suggest that the selected features, whether in simple, multiple, or polynomial regression, are insufficient to accurately model survival time. This highlights the complexity of survival prediction and the need for more informative features or advanced modeling techniques.

3.2.5 Prediction and decision making

```
# prediction and decision making

example_age = pd.DataFrame([[0.5]], columns=['age']) # above-average age (scaled)
predicted_survival = lin_reg.predict(example_age)

print("Predicted survival time (scaled):", predicted_survival)
```

This code demonstrates how the trained simple linear regression model can be used for prediction and decision making. A sample input representing an above-average age value (scaled) is created using a Pandas DataFrame. This input is then passed to the trained regression model using the `predict()` function to estimate the corresponding survival time.

This step illustrates the practical use of the regression model, where new or unseen data can be provided as input to generate predicted outcomes. Such predictions can support data-driven decision making by estimating survival trends based on patient characteristics.

```
Predicted survival time (scaled): [-4.39182603e-05]
```

This code demonstrates how the trained simple linear regression model can be used for prediction and decision making. A sample input representing an above-average age value (scaled) is created using a Pandas DataFrame. This input is then passed to the trained regression model using the `predict()` function to estimate the corresponding survival time.

This step illustrates the practical use of the regression model, where new or unseen data can be provided as input to generate predicted outcomes. Such predictions can support data-driven decision making by estimating survival trends based on patient characteristics.

3.3 Model Evaluation (sklearn)

In this phase, advanced model evaluation and refinement techniques were applied to improve predictive performance and model robustness. Models were evaluated using Scikit-learn tools to assess their generalization ability beyond training data. Issues related to over-fitting and under-fitting were analyzed to guide appropriate model selection.

Regularization techniques such as Ridge Regression were implemented to reduce model complexity and prevent over-fitting. Grid Search was used to systematically tune model hyperparameters and identify optimal configurations. Based on evaluation results, model refinement was performed to enhance performance and ensure better reliability for prediction and decision making.

3.3.1 Model Evaluation

```
# prepare data and train test split

from sklearn.model_selection import train_test_split

X = df_encoded.drop(columns=['survival_status'])
y = df_encoded['survival_status']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In this step, the dataset is prepared for model evaluation by separating the input features and the target variable. The target variable, `survival_status`, is removed from the feature set and stored separately as `y`, while the remaining columns are used as predictors (`X`). The dataset is then split into training and testing sets using an 80:20 ratio. Stratified sampling is applied to ensure that the proportion of survival and non-survival cases remains consistent across both sets. This step is important to ensure a fair and reliable evaluation of the model's performance.

```
# Check for non-numeric columns in X_train
print("Non-numeric columns in X_train:")
print(X_train.select_dtypes(include=['object', 'category']).columns.tolist())
```

```
Non-numeric columns in X_train:
[]
```

Before training the model, a check is performed to identify any non-numeric columns in the training dataset. Since logistic regression requires numerical input, this step ensures that all categorical variables have already been properly encoded. The code selects columns with object or category data types and prints them for verification.

The output shows an empty list, indicating that no non-numeric columns are present in the training data. This confirms that the dataset is fully numeric and suitable for logistic regression, preventing potential errors during model training.

```
# logistic regression

import warnings
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

warnings.filterwarnings('ignore')

log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train)

y_train_pred = log_reg.predict(X_train)
y_test_pred = log_reg.predict(X_test)

print("Logistic Regression (Train)")
print(classification_report(y_train, y_train_pred))

print("Logistic Regression (Test)")
print(classification_report(y_test, y_test_pred))
```

A Logistic Regression model is created and trained using the training dataset. The model is configured with a higher iteration limit to ensure convergence. Once trained, predictions are generated for both the training and testing datasets. The model's performance is then evaluated using a classification report, which includes precision, recall, F1-score, and

accuracy for each class. This allows a detailed assessment of how well the model distinguishes between survival and non-survival cases.

Logistic Regression (Train)				
	precision	recall	f1-score	support
0	0.87	1.00	0.93	34859
1	0.00	0.00	0.00	5137
accuracy			0.87	39996
macro avg	0.44	0.50	0.47	39996
weighted avg	0.76	0.87	0.81	39996
Logistic Regression (Test)				
	precision	recall	f1-score	support
0	0.87	1.00	0.93	8715
1	0.00	0.00	0.00	1285
accuracy			0.87	10000
macro avg	0.44	0.50	0.47	10000
weighted avg	0.76	0.87	0.81	10000

The results show that the model achieves an overall accuracy of approximately 87% on both the training and testing datasets, indicating consistent performance. The model performs very well on the majority class (non-survival), with high precision and recall values. However, the minority class (survival) has very low precision, recall, and F1-score, meaning the model struggles to correctly identify survival cases. This suggests that the dataset is highly imbalanced and that the model is biased toward predicting the majority class. While the accuracy appears high, the poor minority-class performance highlights the need for further improvements such as class balancing, regularization, or alternative model selection strategies.

3.3.2 Over-fitting, under-fitting, and model selection

```
# overfitting and underfitting

from sklearn.metrics import accuracy_score

print("Train accuracy:", accuracy_score(y_train, y_train_pred))
print("Test accuracy :", accuracy_score(y_test, y_test_pred))
```

In this step, the accuracy of the logistic regression model is calculated separately for the training and testing datasets using the `accuracy_score` metric. By comparing these two accuracy values, the model's learning behavior can be assessed. This comparison is a common technique to diagnose whether a model is overfitting (performing much better on training data than test data) or underfitting (performing poorly on both datasets). The code directly prints the training and testing accuracies to allow an easy side-by-side comparison.

```
Train accuracy: 0.8715621562156216
Test accuracy : 0.8715
```

The results show that the training accuracy ($\approx 87.16\%$) and testing accuracy ($\approx 87.15\%$) are almost identical. This indicates that the model generalizes consistently to unseen data and does not suffer from overfitting, as there is no significant performance gap between training and testing results. At the same time, the accuracy is reasonably high, suggesting that the model is not underfitting either. Based on this balance, the logistic regression model can be considered an appropriate choice for this task, making it a suitable candidate for model selection before further refinement techniques such as regularization or grid search are applied.

3.3.3 Ridge Regression

```
# ridge regression

from sklearn.linear_model import RidgeClassifier

ridge = RidgeClassifier(alpha=1.0)
ridge.fit(X_train, y_train)

y_train_pred_ridge = ridge.predict(X_train)
y_test_pred_ridge = ridge.predict(X_test)

print("Ridge Classifier (Train)")
print(classification_report(y_train, y_train_pred_ridge))

print("Ridge Classifier (Test)")
print(classification_report(y_test, y_test_pred_ridge))
```

This code implements Ridge Regression using RidgeClassifier, which applies L2 regularization to reduce model complexity and control overfitting. The classifier is trained using the previously split training data (X_train, y_train) with a regularization strength set by alpha = 1.0. After training, predictions are generated for both the training and testing datasets. The classification_report function is then used to evaluate the model's performance by calculating precision, recall, F1-score, and accuracy for each class. This allows a direct comparison between training and testing performance to assess generalization.

```
Ridge Classifier (Train)
      precision    recall  f1-score   support

     0       0.87        1.00        0.93       34859
     1       0.00        0.00        0.00        5137

 accuracy          0.87          0.87          0.87       39996
 macro avg       0.44        0.50        0.47       39996
weighted avg       0.76        0.87        0.81       39996

Ridge Classifier (Test)
      precision    recall  f1-score   support

     0       0.87        1.00        0.93        8715
     1       0.00        0.00        0.00       1285

 accuracy          0.87          0.87          0.87      10000
 macro avg       0.44        0.50        0.47      10000
weighted avg       0.76        0.87        0.81      10000
```

The output shows that the Ridge Classifier achieves an overall accuracy of 87% on both the training and testing datasets, indicating stable generalization and no significant overfitting. However, the model performs well only for Class 0, with high precision and recall, while Class 1 has zero precision, recall, and F1-score. This indicates that the model is biased toward the majority class and fails to correctly identify the minority class. Although ridge regression helps control overfitting through regularization, the results suggest that class imbalance remains a major issue, and further refinement such as class weighting, resampling, or hyperparameter tuning (e.g., Grid Search) is required to improve minority class prediction.

3.3.4 Grid Search

```
# grid search

from sklearn.model_selection import GridSearchCV

param_grid = {
    'alpha': [0.01, 0.1, 1, 10, 100]
}

grid = GridSearchCV(
    RidgeClassifier(),
    param_grid,
    cv=5,
    scoring='f1'
)

grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)
print("Best CV score:", grid.best_score_)
```

This code applies Grid Search with Cross-Validation using GridSearchCV to optimize the hyperparameter alpha for the RidgeClassifier. A parameter grid is defined with multiple alpha values ranging from 0.01 to 100, allowing the model to test different regularization strengths. The grid search uses 5-fold cross-validation (cv=5) and evaluates performance using the F1-score, which is appropriate for imbalanced datasets. The model is trained on the training data (X_train, y_train), and GridSearchCV automatically tests all parameter combinations to identify the best-performing configuration.

```
Best parameters: {'alpha': 0.01}
Best CV score: 0.0
```

The output shows that the best alpha value is 0.01, indicating that weaker regularization performs slightly better for this dataset. However, the best cross-validation F1-score is 0.0, which suggests that the model still fails to correctly predict the minority class during cross-validation. This result highlights that, although hyperparameter tuning was performed, class imbalance remains a critical limitation, and ridge regression alone is insufficient to

improve minority class performance. Additional techniques such as class weighting, resampling, or alternative models may be required for further refinement.

3.3.5 Model Refinement

```
# model refinement

best_ridge = grid.best_estimator_

y_test_pred_best = best_ridge.predict(X_test)

print("Tuned Ridge (Test)")
print(classification_report(y_test, y_test_pred_best))
```

This code performs model refinement by using the best Ridge Classifier identified from the earlier grid search. The line `best_ridge = grid.best_estimator_` retrieves the model configuration with the optimal hyperparameter (alpha) based on cross-validation. The refined model is then used to make predictions on the test dataset (`X_test`) to evaluate its generalization performance. Finally, a classification report is generated to assess precision, recall, F1-score, and accuracy for each class after tuning.

```
Tuned Ridge (Test)
              precision    recall  f1-score   support

     0       0.87         1.00         0.93         8715
     1       0.00         0.00         0.00          1285

 accuracy          0.87         10000
 macro avg         0.44         0.50         0.47         10000
 weighted avg         0.76         0.87         0.81         10000
```

The output shows that the tuned Ridge model achieves an overall accuracy of 87%, which is consistent with previous models. The model performs very well for class 0 (non-event), with high precision, recall, and F1-score. However, performance for class 1 (event) remains poor, with precision, recall, and F1-score all equal to 0. This indicates that, despite hyperparameter tuning, the refined model still struggles to detect the minority class. Overall, the refinement improves model stability but highlights that class imbalance is a major limitation, suggesting that further techniques such as class weighting, resampling, or alternative models are needed for better minority class prediction.

4.0 Testing and Validation

Model Performance

- Logistic Regression: Accurately predicts the majority class (class 0) but fails to detect the minority class, resulting in an F1-score of 0 for patients who survive.
- Ridge Classifier: Shows the same limitation as Logistic Regression, unable to predict the minority class.
- Random Forest: Perfectly fits the training data (overfitting) but still fails to predict the minority class in the test set.

Validation

- Ridge Classifier Cross-Validation: The F1-score of 0.0 for the minority class indicates poor predictive power for class 1, confirming that model performance is insufficient for the underrepresented group.
- Regression Models: Linear, multiple, and polynomial regressions produce R^2 values near zero, suggesting the selected features are poor predictors of survival time (months).
- ANOVA Analysis: Shows that Stage_at_Diagnosis has a significant impact on survival, with F-statistic = 26,644 and p-value = 0, highlighting it as a key feature.

Future Improvement

- Address Class Imbalance: Use oversampling (e.g., SMOTE), undersampling, or apply balanced class weights in classifiers.
- Advanced Models: Implement Gradient Boosting, Balanced Random Forest, or XGBoost with imbalance handling.
- Evaluation Metrics: Focus on recall, F1-score, and ROC-AUC, rather than accuracy, to better assess minority class prediction.

Key Features

- Stage_at_Diagnosis: Most influential factor affecting survival.
- Treatment_Type: Shows moderate effect on survival time.
- Age & Age_Group: Minor effects based on correlation analysis and basic grouping.

5.0 Conclusion

This project was able to show how bioinformatics and machine learning methods can be used to analyze a large dataset of pancreatic cancer and how these methods can be used to uncover factors linked to patient survival rates. The study discussed a valuable clinical issue, as the pancreatic cancer development is often silent and its diagnosis often late, which makes the model of the survival-related trends based on the clinical, demographic and lifestyle factors sporadic.

Data preprocessing was extensive to provide quality and reliability of the data. The dataset, which comprised about 50,000 records of patients, was cleaned, standardized, and transformed systematically with the help of Python libraries, including Pandas, NumPy, and Scikit-learn. Exploratory Data Analysis (EDA) gave useful insights on the structure of data sets and clinically meaningful trends were found. The descriptive statistics proved that pancreatic cancer occurs mostly in older people and is linked to unfavorable prognoses. The further division into analysis and ANOVA have shown that the stage of diagnosis also significantly affects survival time, which is a strong support of the necessity to detect it as early as possible.

A number of regression-based models were constructed to investigate the association between patient attributes and the survival time. Simple, multiple, and polynomial regression models all had extremely low values of R-squared and proximate values of mean squared error of near one and therefore had little explanatory power. These findings point to the fact that the survival outcomes of pancreatic cancer are complex and multifactorial in their implications and cannot be sufficiently modeled through the simple regression strategies with a restricted set of predictors.

To conduct classification processes, ridge regression and logistic regression models were run to classify survival status. There was good generalization performance indicated by relatively high overall accuracy of 87 percent in both models. Nevertheless, additional analysis showed

that there was a major weakness of the models, which identified the minority classification (survivors) very poorly, especially because of the extreme imbalance of classes in the data. Even though the ridge regression served to mitigate the overfitting with the assistance of regularization, it was not effective enough to enhance the prediction of minority classes. This observation highlights that precision is not a useful measure of unbalanced medical data and that more aggressive appraisal plans are needed.

To sum up, this study has shown the opportunities and the drawbacks of the use of simple machine learning models in predicting pancreatic cancer. Although the analysis was able to detect significant trends, especially that cancer stage is a major predictor of survival, the predictive accuracy of the models is limited due to the selection of features, imbalanced classes, and simplicity of the models. Further research in the area might involve inclusion of more clinical biomarkers, the use of more modern machine learning methods, like ensemble methods or neural networks, and the treatment of class imbalance using resampling or cost-sensitive learning. In general, the given project gives the grounds to continue bioinformatics-based studies to enhance the early diagnosis and prognosis of pancreatic cancer.